

# Shikaku

Sofia Guerra Jiménez<sup>a</sup>

<sup>a</sup>*Pontificia Universidad Javeriana, Facultad de Ingeniería, Departamento de Ingeniería de Sistemas*

---

## Abstract

En este documento se presenta la escritura formal del algoritmo necesario para implementar un jugador automático del juego Buscaminas, basado en un modelo probabilístico de estimación de minas. Se describen el análisis del problema, la clasificación de su complejidad computacional, el diseño de entradas y salidas, el algoritmo formal, su estrategia algorítmica y su análisis de complejidad.

*Keywords:* Shikaku, algoritmo, NP-Completo, combinatoria, heurística, problema de decisión.

---

## 1. Análisis del problema

Shikaku es un rompecabezas que se desarrolla en un tablero de  $W \times H$  celdas. En el tablero hay celdas que contienen un número entero positivo. La suma de estos números equivale al área del rompecabezas, es decir,  $W \times H$ . El objetivo del juego es dividir el tablero en rectángulos de manera que cada rectángulo contenga exactamente un número, que definirá su área. Todas las celdas pertenecen exactamente a un rectángulo.

### 1.1. Clasificación del problema

El problema de Shikaku es un problema de decisión. Se puede basar en la siguiente pregunta: ¿Existe una partición válida para el tablero? O al escoger un rectángulo, ¿El rectángulo puede ir en esa posición?

El problema se puede clasificar como NP-Completo. Para que un problema sea NP-Completo, debe ser al menos tan difícil como un problema NP, y debe ser verificable en tiempo polinomial.

La solución del tablero de Shikaku puede ser verificable en tiempo polinomial, recorriendo el tablero una vez.

### 1.2. Análisis previo de complejidad: Fuerza bruta vs. Heurística

Antes de empezar a implementar la estrategia algorítmica, se debe saber cuál será la aproximación a esta. Puedo realizar el algoritmo por fuerza bruta, o buscar una heurística que Debo realizarme la pregunta

---

*Email address:* [sofiaguerraj@javeriana.edu.co](mailto:sofiaguerraj@javeriana.edu.co) (Sofia Guerra Jiménez)

### 1.2.1. Fuerza bruta

Sea  $\mathbf{T} = W \times H$  el número total de celdas,  $\mathbf{n}$  la cantidad de números enteros en el tablero, y  $k_i$  el valor de cada uno. Por definición del problema, la suma de todas esos valores cubre exactamente el tablero:

$$T = k_1 + k_2 + k_3 + \dots + k_n$$

Para analizar el problema por fuerza bruta, podemos tomar una aproximación por combinatoria.

Para comenzar, de las  $T$  celdas disponibles, escogemos el primer valor  $k_1$ , y buscamos todas las combinaciones posibles de rectángulos válidos con  $k_1$  celdas, y así sucesivamente.

Sin tener en cuenta ninguna de las restricciones del rompecabezas, el problema combinatorio se ve así:

$$C_{total} = C(T, k_1) \times C(T - k_1, k_2) \times C(T - k_1 - k_2, k_3) \times \dots \times C(k_n, k_n)$$

### 1.2.2. Heurística

## 2. Diseño del problema

### 2.1. Entradas

- $W \in \mathbb{N}$ : Ancho del tablero (columnas)
- $H \in \mathbb{N}$ : Alto del tablero (filas)
- Matriz de enteros  $W \times H$ , donde cada celda puede ser un número puede contener un número entero positivo  $k$ , o  $-1$  si no tiene valor. Tenemos la garantía que la suma de los números enteros positivos del tablero serán iguales a  $W \times H$ , o el área del tablero.

### 2.2. Salidas

En cada turno, el jugador produce:

- SALIDAS

## 3. Algoritmo de solución

ALGORITMO DE SOLUCION

## 3.1. Estrategia algorítmica

El solucionador de Shikaku puede estrategia por fuerza bruta

**Algorithm 1** ALGORITMO

---

**Require:** REQUERIMIENTOS

```

1: procedure ALGORITMOSHIKAKU
2:   for each  $(r, c)$  in tablero do
3:     if  $discovered[r][c]$  then  $prob[r][c] \leftarrow 0$ 
4:     else if  $flagged[r][c]$  then  $prob[r][c] \leftarrow 1$ 
5:     else  $prob[r][c] \leftarrow p_{base}$ 
6:     end if
7:   end for
8: end procedure

```

---

## 3.2. Análisis de complejidad

ANALISIS DE COMPLEJIDAD

## 3.3. Notas de implementación

A continuación se presentan los requisitos y consideraciones necesarios para implementar el algoritmo:

- NOTA DE IMPLEMENTACION

## 4. Diseño de la interfaz